

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 Cyber & Information Security Practical

PRACTICAL - 5

Aim :- To implement the Diffie-Hellman Key Exchange algorithm to securely exchange a secret key between two entities over an insecure network using CLI.

Code :-

```
import math
# Function to check whether a number is prime
def is_prime(number):
    if number < 2:
        return False
    for i in range(2, int(math.sqrt(number)) + 1):
        if number % i == 0:
            return False
    return True

# ----- MAIN PROGRAM -----
print("=" * 60)
print("    DIFFIE-HELLMAN KEY EXCHANGE")
print("=" * 60)
print("\nPublic parameters are shared between User 1 and User 2.")
print("Private keys must remain secret.\n")

# ----- INPUT p -----
while True:
    try:
        p = int(input("Enter a prime number (p): "))

        if is_prime(p):
            break
        else:
            print("Error: p must be a prime number.")
    except ValueError:
        print("Error: Please enter a valid integer.")

# ----- INPUT g -----
while True:
    try:
        g = int(input("Enter generator (g): "))
        if 1 < g < p:
            break
        else:
            print("Error: g must satisfy 1 < g < p.")
    except ValueError:
        print("Error: Please enter a valid integer.")

# ----- USER 1 PRIVATE KEY -----
while True:
    try:
        user1_private = int(
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 Cyber & Information Security Practical

```
    input("\nEnter User 1 private key: ")
)
if user1_private > 0:
    break
else:
    print("Private key must be greater than 0.")
except ValueError:
    print("Error: Please enter a valid integer.")

# ----- USER 2 PRIVATE KEY -----
while True:
    try:
        user2_private = int(
            input("Enter User 2 private key: ")
        )
        if user2_private > 0:
            break
        else:
            print("Private key must be greater than 0.")
    except ValueError:
        print("Error: Please enter a valid integer.")

# ----- PUBLIC KEY CALCULATION -----
user1_public = pow(
    g,
    user1_private,
    p
)
user2_public = pow(
    g,
    user2_private,
    p
)

# ----- DISPLAY PUBLIC INFORMATION -----
print("\n" + "=" * 60)
print("PUBLIC PARAMETERS")
print("=" * 60)
print("Prime number (p):", p)
print("Generator (g) :", g)

# ----- DISPLAY PUBLIC KEYS -----
print("\n" + "=" * 60)
print("PUBLIC KEY GENERATION")
print("=" * 60)
print("User 1 Public Key:", user1_public)
print("User 2 Public Key:", user2_public)

# ----- SHARED SECRET CALCULATION -----
user1_shared_secret = pow(
    user2_public,
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 Cyber & Information Security Practical

```
user1_private,
p
)
user2_shared_secret = pow(
    user1_public,
    user2_private,
    p
)

# ----- DISPLAY SHARED KEYS -----
print("\n" + "=" * 60)
print("SHARED SECRET KEY CALCULATION")
print("=" * 60)
print("User 1 calculates:")
print(
    "Shared Key = User 2 Public Key ^ User 1 Private Key mod p"
)
print(
    "User 1 Shared Secret Key:",
    user1_shared_secret
)
print("\nUser 2 calculates:")
print(
    "Shared Key = User 1 Public Key ^ User 2 Private Key mod p"
)
print(
    "User 2 Shared Secret Key:",
    user2_shared_secret
)

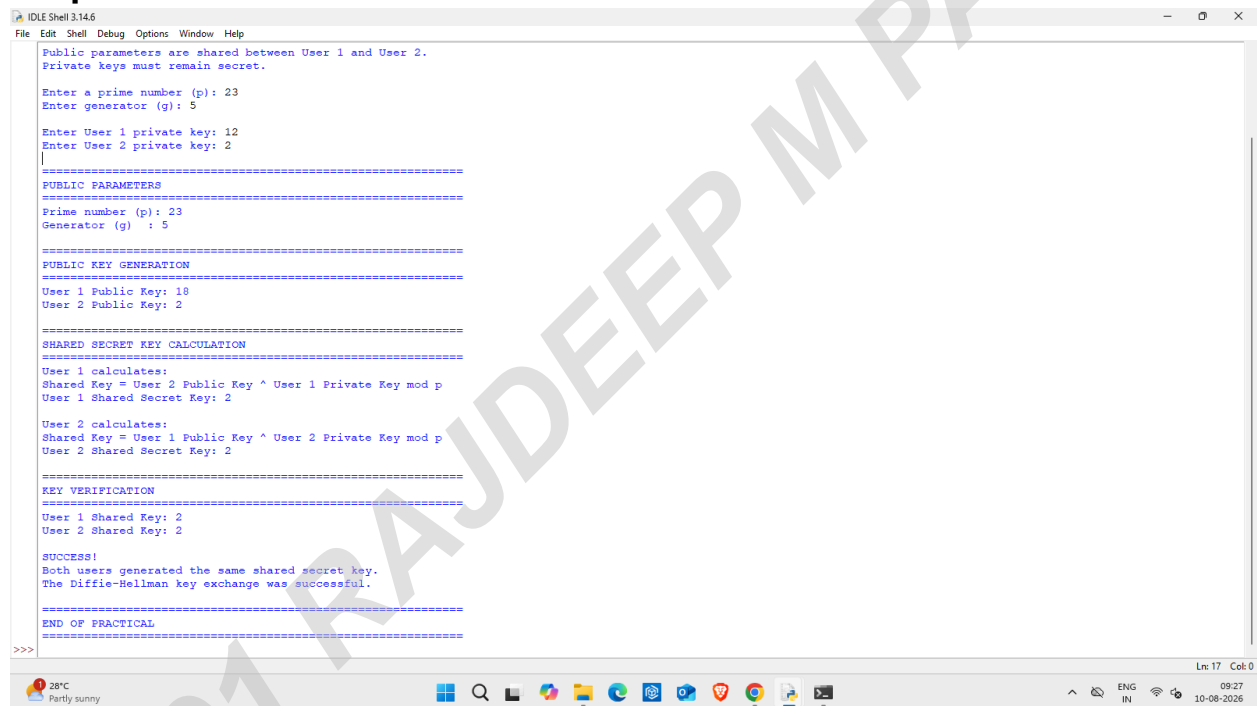
# ----- VERIFICATION -----
print("\n" + "=" * 60)
print("KEY VERIFICATION")
print("=" * 60)
if user1_shared_secret == user2_shared_secret:
    print(
        "User 1 Shared Key:",
        user1_shared_secret
    )
    print(
        "User 2 Shared Key:",
        user2_shared_secret
    )
    print("\nSUCCESS!")
    print(
        "Both users generated the same shared secret key."
    )
    print(
        "The Diffie-Hellman key exchange was successful."
    )
else:
    print(
        "User 1 Shared Key:",
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 Cyber & Information Security Practical

```
user1_shared_secret
)
print(
    "User 2 Shared Key:",
    user2_shared_secret
)
print("\nFAILED!")
print(
    "The shared keys are different."
)
print("\n" + "=" * 60)
print("END OF PRACTICAL")
print("=" * 60)
```

Output :-



```
IDLE Shell 3.14.6
File Edit Shell Debug Options Window Help

Public parameters are shared between User 1 and User 2.
Private keys must remain secret.

Enter a prime number (p): 23
Enter generator (g): 5

Enter User 1 private key: 12
Enter User 2 private key: 2
|
=====
PUBLIC PARAMETERS
=====
Prime number (p): 23
Generator (g) : 5

=====
PUBLIC KEY GENERATION
=====
User 1 Public Key: 18
User 2 Public Key: 2

=====
SHARED SECRET KEY CALCULATION
=====
User 1 calculates:
Shared Key = User 2 Public Key ^ User 1 Private Key mod p
User 1 Shared Secret Key: 2

User 2 calculates:
Shared Key = User 1 Public Key ^ User 2 Private Key mod p
User 2 Shared Secret Key: 2

=====
KEY VERIFICATION
=====
User 1 Shared Key: 2
User 2 Shared Key: 2

SUCCESS!
Both users generated the same shared secret key.
The Diffie-Hellman key exchange was successful.

=====
END OF PRACTICAL
=====
>>>
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 Cyber & Information Security Practical

Aim :- To implement the Diffie-Hellman Key Exchange algorithm to securely exchange a secret key between two entities over an insecure network using CLI.

Code :-

```
import tkinter as tk
from tkinter import messagebox
import math

# ----- PRIME NUMBER CHECK -----
def is_prime(number):
    if number < 2:
        return False
    for i in range(2, int(math.sqrt(number)) + 1):
        if number % i == 0:
            return False
    return True

# ----- DIFFIE-HELLMAN CALCULATION -----
def calculate_keys():
    try:
        # Get values from GUI
        p = int(p_entry.get())
        g = int(g_entry.get())
        user1_private = int(user1_private_entry.get())
        user2_private = int(user2_private_entry.get())
        # Validate p
        if not is_prime(p):
            messagebox.showerror(
                "Invalid Input",
                "p must be a prime number."
            )
            return
        # Validate g
        if g <= 1 or g >= p:
            messagebox.showerror(
                "Invalid Input",
                "g must satisfy 1 < g < p."
            )
            return
        # Validate private keys
        if user1_private <= 0:
            messagebox.showerror(
                "Invalid Input",
                "User 1 private key must be greater than 0."
            )
            return
        if user2_private <= 0:
            messagebox.showerror(
                "Invalid Input",
                "User 2 private key must be greater than 0."
            )
            return
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 Cyber & Information Security Practical

```
)
return

# ----- PUBLIC KEYS -----
user1_public = pow(
    g,
    user1_private,
    p
)
user2_public = pow(
    g,
    user2_private,
    p
)

# ----- SHARED SECRET KEYS -----
user1_shared_secret = pow(
    user2_public,
    user1_private,
    p
)
user2_shared_secret = pow(
    user1_public,
    user2_private,
    p
)

# ----- DISPLAY OUTPUT -----
output_text.delete(
    "1.0",
    tk.END
)
output_text.insert(
    tk.END,
    "=" * 65 + "\n"
)
output_text.insert(
    tk.END,
    "    DIFFIE-HELLMAN KEY EXCHANGE RESULT\n"
)
output_text.insert(
    tk.END,
    "=" * 65 + "\n\n"
)
output_text.insert(
    tk.END,
    "PUBLIC PARAMETERS\n"
)
output_text.insert(
    tk.END,
    "-" * 65 + "\n"
)
output_text.insert(
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 Cyber & Information Security Practical

```
tk.END,  
f"Prime Number (p): {p}\n"  
)  
output_text.insert(  
tk.END,  
f"Generator (g) : {g}\n\n"  
)  
output_text.insert(  
tk.END,  
"PRIVATE KEYS\n"  
)  
output_text.insert(  
tk.END,  
"- * 65 + "\n"  
)  
output_text.insert(  
tk.END,  
f"User 1 Private Key: {user1_private}\n"  
)  
output_text.insert(  
tk.END,  
f"User 2 Private Key: {user2_private}\n\n"  
)  
output_text.insert(  
tk.END,  
"PUBLIC KEY GENERATION\n"  
)  
  
output_text.insert(  
tk.END,  
"- * 65 + "\n"  
)  
output_text.insert(  
tk.END,  
f"User 1 Public Key =  $g^{\text{User1\_Private}} \bmod p$ \n"  
)  
  
output_text.insert(  
tk.END,  
f"User 1 Public Key = {user1_public}\n\n"  
)  
output_text.insert(  
tk.END,  
f"User 2 Public Key =  $g^{\text{User2\_Private}} \bmod p$ \n"  
)  
output_text.insert(  
tk.END,  
f"User 2 Public Key = {user2_public}\n\n"  
)  
output_text.insert(  
tk.END,  
"SHARED SECRET KEY CALCULATION\n"  
)
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 Cyber & Information Security Practical

```
output_text.insert(
    tk.END,
    "-" * 65 + "\n"
)
output_text.insert(
    tk.END,
    "User 1 calculates:\n"
)
output_text.insert(
    tk.END,
    "Shared Key = User 2 Public Key ^ User 1 Private Key mod p\n"
)
output_text.insert(
    tk.END,
    f"User 1 Shared Secret Key = {user1_shared_secret}\n\n"
)
output_text.insert(
    tk.END,
    "User 2 calculates:\n"
)
output_text.insert(
    tk.END,
    "Shared Key = User 1 Public Key ^ User 2 Private Key mod p\n"
)
output_text.insert(
    tk.END,
    f"User 2 Shared Secret Key = {user2_shared_secret}\n\n"
)
output_text.insert(
    tk.END,
    "KEY VERIFICATION\n"
)
output_text.insert(
    tk.END,
    "-" * 65 + "\n"
)

# ----- VERIFICATION -----
if user1_shared_secret == user2_shared_secret:
    output_text.insert(
        tk.END,
        "STATUS: SUCCESS\n\n"
    )
    output_text.insert(
        tk.END,
        "Both users generated the same shared secret key.\n"
    )
    output_text.insert(
        tk.END,
        "The Diffie-Hellman key exchange was successful.\n"
    )
    messagebox.showinfo(
```


SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 Cyber & Information Security Practical

```
        "Success",
        "Diffie-Hellman Key Exchange Successful!\n\n"
        f"Shared Secret Key: {user1_shared_secret}"
    )
else:
    output_text.insert(
        tk.END,
        "STATUS: FAILED\n\n"
    )
    output_text.insert(
        tk.END,
        "The shared keys are different.\n"
    )
    output_text.insert(
        tk.END,
        "The Diffie-Hellman key exchange failed.\n"
    )
    messagebox.showerror(
        "Failed",
        "Diffie-Hellman Key Exchange Failed!"
    )
except ValueError:
    messagebox.showerror(
        "Invalid Input",
        "Please enter valid integer values."
    )

# ----- CLEAR FUNCTION -----
def clear_all():
    p_entry.delete(
        0,
        tk.END
    )
    g_entry.delete(
        0,
        tk.END
    )
    user1_private_entry.delete(
        0,
        tk.END
    )
    user2_private_entry.delete(
        0,
        tk.END
    )
    output_text.delete(
        "1.0",
        tk.END
    )

# ----- MAIN WINDOW -----
root = tk.Tk()
root.title(
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 Cyber & Information Security Practical

"Cybersecurity Practical 5 - Diffie-Hellman Key Exchange"

```
)
root.geometry(
    "900x750"
)
root.resizable(
    True,
    True
)

# ----- TITLE -----
title_label = tk.Label(
    root,
    text="DIFFIE-HELLMAN KEY EXCHANGE",
    font=("Arial", 20, "bold")
)
title_label.pack(
    pady=10
)
subtitle_label = tk.Label(
    root,
    text="Cybersecurity Practical 5",
    font=("Arial", 12)
)
subtitle_label.pack(
    pady=2
)

# ----- INPUT FRAME -----
input_frame = tk.LabelFrame(
    root,
    text="User Input",
    font=("Arial", 12, "bold"),
    padx=15,
    pady=15
)
input_frame.pack(
    fill="x",
    padx=25,
    pady=15
)

# p
tk.Label(
    input_frame,
    text="Prime Number (p):",
    font=("Arial", 11)
).grid(
    row=0,
    column=0,
    padx=10,
    pady=8,
    sticky="w"
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 Cyber & Information Security Practical

```
)
p_entry = tk.Entry(
    input_frame,
    width=25,
    font=("Arial", 11)
)
p_entry.grid(
    row=0,
    column=1,
    padx=10,
    pady=8
)
# g
tk.Label(
    input_frame,
    text="Generator (g):",
    font=("Arial", 11)
).grid(
    row=1,
    column=0,
    padx=10,
    pady=8,
    sticky="w"
)
g_entry = tk.Entry(
    input_frame,
    width=25,
    font=("Arial", 11)
)
g_entry.grid(
    row=1,
    column=1,
    padx=10,
    pady=8
)

# User 1 Private Key
tk.Label(
    input_frame,
    text="User 1 Private Key:",
    font=("Arial", 11)
).grid(
    row=2,
    column=0,
    padx=10,
    pady=8,
    sticky="w"
)
user1_private_entry = tk.Entry(
    input_frame,
    width=25,
    font=("Arial", 11)
)
```

T101

RAJDEEP M PARAB

TY.BSc.CS

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 Cyber & Information Security Practical

```
user1_private_entry.grid(
    row=2,
    column=1,
    padx=10,
    pady=8
)

# User 2 Private Key
tk.Label(
    input_frame,
    text="User 2 Private Key:",
    font=("Arial", 11)
).grid(
    row=3,
    column=0,
    padx=10,
    pady=8,
    sticky="w"
)

user2_private_entry = tk.Entry(
    input_frame,
    width=25,
    font=("Arial", 11)
)

user2_private_entry.grid(
    row=3,
    column=1,
    padx=10,
    pady=8
)

# ----- BUTTONS -----
button_frame = tk.Frame(
    root
)

button_frame.pack(
    pady=5
)

calculate_button = tk.Button(
    button_frame,
    text="Calculate Key Exchange",
    command=calculate_keys,
    width=25,
    font=("Arial", 11, "bold")
)

calculate_button.pack(
    side="left",
    padx=10
)

clear_button = tk.Button(
    button_frame,
    text="Clear",
    command=clear_all,
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 Cyber & Information Security Practical

```
width=15,
font=("Arial", 11)
)
clear_button.pack(
    side="left",
    padx=10
)
exit_button = tk.Button(
    button_frame,
    text="Exit",
    command=root.destroy,
    width=15,
    font=("Arial", 11)
)
exit_button.pack(
    side="left",
    padx=10
)

# ----- OUTPUT FRAME -----
output_frame = tk.LabelFrame(
    root,
    text="Output",
    font=("Arial", 12, "bold"),
    padx=10,
    pady=10
)
output_frame.pack(
    fill="both",
    expand=True,
    padx=25,
    pady=15
)
output_text = tk.Text(
    output_frame,
    height=25,
    width=100,
    font=("Consolas", 10)
)
output_text.pack(
    fill="both",
    expand=True
)

# ----- START GUI -----
root.mainloop()
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 Cyber & Information Security Practical

Output :-

The screenshot shows a Windows application titled "DIFFIE-HELLMAN KEY EXCHANGE" with a subtitle "Cybersecurity Practical 5". The application has a "User Input" section with four text boxes: "Prime Number (p):" with value 23, "Generator (g):" with value 5, "User 1 Private Key:" with value 12, and "User 2 Private Key:" with value 2. Below the input section is an "Output" section displaying the results of the key exchange. A "Calculate Key Exchange" button is visible, and a "Success" dialog box is open in the center, displaying "Diffie-Hellman Key Exchange Successful!" and "Shared Secret Key: 2".

DIFFIE-HELLMAN KEY EXCHANGE
Cybersecurity Practical 5

User Input

Prime Number (p): 23
Generator (g): 5
User 1 Private Key: 12
User 2 Private Key: 2

Output

DIFFIE-HELLMAN KEY EXCHANGE RESULT

PUBLIC PARAMETERS
Prime Number (p): 23
Generator (g) : 5

PRIVATE KEYS
User 1 Private Key: 12
User 2 Private Key: 2

PUBLIC KEY GENERATION
User 1 Public Key = $g^{\text{User1_Private}} \bmod p$
User 1 Public Key = 18
User 2 Public Key = $g^{\text{User2_Private}} \bmod p$
User 2 Public Key = 2

SHARED SECRET KEY CALCULATION
User 1 calculates:
Shared Key = User 2 Public Key ^ User 1 Private Key mod p

Success
Diffie-Hellman Key Exchange Successful!
Shared Secret Key: 2